



Informe Técnico: Justificación del Stack — NominaAI

Proyecto: Extractor inteligente de nóminas con OCR + LLM + validación de datos
Fecha: 15 de julio de 2026
Autor: Andrés Torrez
Destinatario: Kontaktu AI (Miguel Álvarez Mulas) — Challenge Técnico
Motor OCR seleccionado: Groq (Llama 4 Scout)



Resumen Ejecutivo

Este informe documenta la justificación técnica de cada tecnología seleccionada para el proyecto **NominaAI**. El objetivo no es solo construir un extractor de nóminas funcional, sino demostrar **cómo trabajamos con IA, cómo manejamos que el LLM a veces falla, y cómo validamos resultados** antes de presentarlos al usuario. Cada elección de stack responde a una pregunta concreta: ¿es rápido? ¿es gratuito? ¿escala? ¿es fácil de explicar en una entrevista de 30 minutos?

Decisión clave: Elegimos **Groq (Llama 4 Scout)** como motor de OCR por su velocidad ultra-rápida (500-3,000 tokens/segundo), gratuidad absoluta, y capacidad de visión multimodal. La velocidad es un factor diferenciador en una demo de 30 minutos: el evaluador ve resultados en tiempo real, no espera 5 segundos mirando una pantalla de carga.



Stack Tecnológico Completo

Capa	Tecnología	Rol en el proyecto
IDE	Cursor	Editor con IA integrada para iterar rápido
Asistente IA	Kimi Claw	Automatización de tareas repetitivas
Backend	Python + FastAPI	API REST moderna, async, con documentación automática
Frontend	Next.js 14 + Tailwind + shadcn/ui	UI profesional, componentes pre-diseñados, SSR opcional
LLM / OCR	Groq (Llama 4 Scout)	Visión + extracción estructurada, gratis, ultra-rápido, OpenAI-compatible
Base de datos	SQLite	Zero-config, perfecto para prototipos y demos
Control de versiones	Git + GitHub	Repo + Projects (Kanban) + PRs limpios
Validación	Pydantic + Reglas propias	Typed data + lógica de negocio para detectar errores del LLM
Estructuración de salida	Instructor	Forzar JSON estricto desde el LLM usando Pydantic



1. Groq (Llama 4 Scout) — El Motor de OCR

¿Por qué Groq y no GitHub Models ni OpenAI?

Factor	OpenAI (GPT-4o)	GitHub Models (GPT-4o)	Groq (Llama 4 Scout)
Costo	Pago desde el primer token (~\$0.005/imagen)	Gratis (limitado a ~500-1,000 RPD)	Gratis (14,400 RPD / 30,000 TPM)
Tarjeta de crédito	Requerida	No requerida	No requerida
Velocidad	~30-100 tokens/seg	~30-100 tokens/seg	⚡ 500-3,000+ tokens/seg (5-30x más rápido)
Calidad OCR	🥇 Mejor del mundo	🥇 Mejor del mundo	🟡 Buena (suficiente para nóminas)
Nóminas españolas	Excelente	Excelente	Buena (aceptable con validación)
JSON estructurado	Muy obediente	Muy obediente	Obediente (con prompt bien diseñado)
Setup	API key + billing	Token de GitHub	API key de Groq (muy fácil)
Compatibilidad API	OpenAI nativo	OpenAI-compatible (Azure)	OpenAI-compatible
Vendor lock-in	Sí (propietario)	No (portable)	No (open-source, migrable)
Demo de 30 min	Lento, costoso	Lento, gratuito	⚡ Rápido, gratuito, impresionante

¿Por qué la velocidad gana en este challenge?

- 1. **Demo en tiempo real:** En una entrevista de 30 minutos, subes una nómina y el resultado aparece en **0.3-1 segundo**. Eso impresiona. Con GPT-4o esperas 3-5 segundos. El evaluador mira la pantalla en silencio. Eso mata el ritmo.
- 2. **Iteración del prompt:** Si necesitas ajustar el prompt 10 veces durante el desarrollo, Groq te ahorra horas. Cada iteración es instantánea.
- 3. **Reintentos en vivo:** Si el LLM falla una extracción, con Groq el reintento es inmediato. Con GPT-4o, cada reintento son 5 segundos de silencio incómodo.
- 4. **Narrativa de la entrevista:** “Usé Groq porque es 30 veces más rápido que la API estándar de OpenAI. Esto permite que el usuario vea resultados en tiempo real, sin fricción. El código está diseñado para ser portable: cambio `base_url` y `model`, y funciona con cualquier proveedor OpenAI-compatible.” Eso demuestra pensamiento de arquitectura y experiencia de usuario.

¿Por qué aceptamos menor precisión OCR?

Porque **la validación compensa la precisión**. Si Llama 4 Scout confunde un dígito del IBAN (8 con B), nuestro validador lo detecta, lo marca como `confidence: low`, y el usuario lo corrige. El flujo completo sigue siendo robusto. La velocidad del LLM + la rigurosidad del validador = producto mejor que un LLM lento pero perfecto sin validación.

Modelo específico

llama-4-scout-17b-16e-instruct

- Capacidades: visión multimodal, tool use, JSON mode, multilingual
- Contexto: 128K tokens
- Límite de imagen: 20MB (URL) / 4MB (base64) / 33 megapíxeles
- Máximo 5 imágenes por request
- Free tier: 14,400 requests/day, 30,000 tokens/minute

Flujo de OCR con Groq

```
PDF/Image → Convertir a base64 → Enviar a Groq (Llama 4 Scout)
↓
Prompt estructurado con Pydantic schema (vía Instructor)
↓
JSON estricto con: nombre, empresa, ingresos, fecha, IBAN
↓
Validador propio detecta errores del LLM
↓
Guardar en SQLite con flags de confianza
```

Código de conexión

```
from groq import Groq

client = Groq(
    api_key="TU_GROQ_API_KEY" # console.groq.com → Create API Key
)

response = client.chat.completions.create(
    model="llama-4-scout-17b-16e-instruct",
    messages=[{
        "role": "user",
        "content": [
            {"type": "text", "text": "Extrae los datos de esta nómina..."},
            {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{base64_image}"}}
        ]
    }]
)
```

¿Por qué no Instructor con Groq?

Instructor funciona nativamente con clientes OpenAI. Para Groq, usamos el cliente `groq` directamente con `response_format={"type": "json_object"}` para forzar JSON. Si Groq no soporta `response_format` en el modelo de visión, parseamos manualmente con Pydantic y reintentamos si falla. Esto es una decisión consciente: preferimos velocidad Groq + parsing manual robusto, sobre Instructor + OpenAI lento.

2. ⚡ FastAPI — El Backend

¿Por qué FastAPI y no Flask o Django?

Factor	Flask	Django	FastAPI
Async nativo	No (requiere extensiones)	No	Sí (async/await nativo)
Documentación auto	No (requiere Swagger manual)	Parcial	Sí (/docs automático)
Validación de datos	Manual	Forms	Pydantic integrado
Performance	Media	Media	Alta (Starlette + Uvicorn)
Curva de aprendizaje	Baja	Alta	Media
Tamaño del proyecto	Cualquiera	Grande	Medio/pequeño

Justificación

- Async nativo:** Subir archivos, llamar a Groq, validar datos — todo puede ser async. En Flask tendríamos que usar threads o Celery. En FastAPI es `async def` y listo.
- Documentación automática:** El endpoint `/docs` genera una UI Swagger interactiva. En la entrevista, puedo mostrar: “Aquí está mi API documentada, probada y lista para integrar.” Eso es profesional sin esfuerzo extra.
- Pydantic integrado:** La validación de datos del backend y la validación del LLM usan la misma librería. Un solo modelo Pydantic define el schema de extracción, y se usa tanto para parsear el JSON del LLM como para validar la respuesta.
- Moderno y estándar:** FastAPI es el framework Python más usado en 2026 para APIs nuevas. Usarlo demuestra que estamos al día, no pegados a stacks de hace 5 años.

3. 🌈 Next.js 14 + Tailwind + shadcn/ui — El Frontend

¿Por qué este stack frontend?

Factor	React vanilla	Next.js 14 + shadcn/ui
Componentes UI	Manual o librerías sueltas	shadcn/ui: 40+ componentes copiados, no instalados
Estilos	CSS modules / styled-components	Tailwind: utility-first, rápido, consistente
SSR / SEO	No	Sí (App Router)
Rutas	React Router manual	File-based routing automático
Performance	Media	Alta (optimizaciones automáticas)
Tiempo de setup	Alto	Bajo (<code>npx shadcn@latest init</code>)

Justificación

- shadcn/ui = componentes profesionales sin dependencias:** No instalamos una librería de componentes. Copiamos el código de cada componente a nuestro repo. Eso significa: control total,

personalización ilimitada, y cero conflictos de versiones. En una entrevista: “Uso shaden porque cada componente es mío, no una caja negra.”

2. **Tailwind = velocidad de diseño:** No escribimos CSS. Escribimos clases. `bg-red-500`, `rounded-lg`, `shadow-md`. En 30 minutos de challenge, eso es la diferencia entre una UI fea y una UI que parece producto real.
3. **Next.js 14 App Router:** Rutas basadas en archivos. `app/result/[file_id]/page.tsx` = ruta dinámica automática. No configuramos router. No perdemos tiempo.
4. **Primera impresión:** La UI es lo primero que ven en la demo. Si el dropzone brilla, el dashboard es limpio, y los estados de error son amigables, el evaluador ya está convencido antes de ver el código.

4. 🗄️ SQLite — La Base de Datos

¿Por qué SQLite y no PostgreSQL o MongoDB?

Factor	PostgreSQL	MongoDB	SQLite
Setup	Docker + config	Servidor + config	Zero-config (archivo <code>.db</code>)
Dependencias	psycopg2, Docker	pymongo, servidor	sqlite3 (incluido en Python)
Portabilidad	Media	Media	Alta (archivo único)
Escalabilidad	Alta	Alta	Media (suficiente para demo)
ORM	SQLAlchemy	Motor	SQLAlchemy (mismo ORM)

Justificación

1. **Zero-config:** No necesitamos Docker, no necesitamos instalar PostgreSQL, no necesitamos crear usuarios ni passwords. `sqlite:///./nomina.db` y listo. En un challenge de 30-60 minutos, cada minuto de setup ahorrado es un minuto de código ganado.
2. **Portabilidad:** La base de datos es un archivo `.db`. Puedo enviarlo por email, subirlo a GitHub (con `.gitignore`), o mostrarlo en la entrevista. “Aquí está mi base de datos, la abro con DB Browser y ves los datos reales.”
3. **SQLAlchemy funciona igual:** Si en producción se migra a PostgreSQL, cambiamos una línea: `sqlite:///./nomina.db` → `postgresql://....`. El resto del código es idéntico. Demuestra que pensamos en escalabilidad desde el principio.
4. **Suficiente para el challenge:** Una tabla `extractions` con 10-20 campos. SQLite maneja eso sin problemas. No over-engineering.

5. 🎯 Pydantic — Estructuración y Validación de Salida

¿Por qué Pydantic sin Instructor?

Instructor es una librería que “parchea” el cliente de OpenAI para forzar JSON estricto. Como usamos **Groq** (cliente groq, no openai), Instructor no se integra directamente. En su lugar, usamos **Pydantic** de dos maneras:

1. **Parseo manual robusto:** Definimos el schema Pydantic (`NominaExtraction`), y después de recibir la respuesta del LLM, intentamos parsearla. Si falla, capturamos el error, loggeamos la respuesta cruda, y reintentamos con un prompt más estricto.
2. **Validación de negocio:** El mismo modelo Pydantic se usa para:
 - Parsear el JSON del LLM
 - Validar la respuesta en el backend (FastAPI)
 - Tipar la respuesta de la API
 - Definir el schema de la base de datos (SQLAlchemy)

Ventaja de esta aproximación

- **Más control:** No dependemos de una librería externa (Instructor) para forzar el formato. Nuestro código maneja los fallos explícitamente.
- **Narrativa de la entrevista:** “No uso Instructor porque trabajo con Groq. En su lugar, diseñé un parser Pydantic con reintentos automáticos y logging de fallos. Así veo exactamente cuándo y por qué el LLM se equivoca, y puedo ajustar el prompt.” Eso demuestra que entiendes el problema, no solo que copias una librería.

6. 🔍 Validador Propio — Cuando la IA Miente

¿Por qué un validador propio y no confiar en el LLM?

El LLM puede:

- Leer mal un dígito del IBAN (confundir 8 con B)
- Invertir brutos y netos
- Inventar una fecha si no la encuentra
- Dejar campos vacíos sin advertir

Reglas de validación implementadas

Campo	Regla	Qué pasa si falla
IBAN	Longitud 24, empieza por ES, checksum válido	<code>confidence: low</code> , flag: “IBAN inválido”
Ingresos netos	Deben ser < ingresos brutos	<code>confidence: low</code> , flag: “Incoherencia lógica”

Table 6 – continued

Campo	Regla	Qué pasa si falla
Fecha	Formato válido, no futura, no anterior a 2020	<code>confidence: low</code> , flag: “Fecha inválida”
Nombre	No vacío, al menos 2 palabras	<code>confidence: low</code> , flag: “Nombre incompleto”
Empresa	No vacío, >2 caracteres	<code>confidence: low</code> , flag: “Empresa no detectada”

Justificación

1. **No rechazamos todo:** Si el IBAN falla pero el resto está bien, mostramos los datos válidos y resaltamos solo el IBAN. El usuario corrige solo lo que está mal. Eso es UX de producto real, no “todo o nada”.
2. **Auditoría:** Guardamos `raw_llm_response` (lo que el LLM devolvió crudo) y `validation_flags` (qué falló). Si hay una disputa, podemos revisar qué dijo la IA y por qué lo flaggeamos.
3. **Narrativa de la entrevista:** “El LLM me devolvió un IBAN que empezaba por ‘ES91’ pero el checksum no cuadraba. Mi validador lo detectó, lo marcó como dudoso, y el usuario lo corrigió. Eso es lo que evalúan: no que funcione perfecto, sino cómo manejamos los fallos.”

7. Cursor — El IDE

¿Por qué Cursor y no VS Code vanilla?

1. **IA integrada en el editor:** Cursor tiene Claude/GPT integrado en el propio IDE. Puedo seleccionar código, presionar `Ctrl+K`, y pedir “refactoriza esto a `async`”. La IA entiende el contexto del archivo abierto y del proyecto entero.
2. **Composer:** Cursor puede generar múltiples archivos a la vez. “Crea el endpoint de upload, el servicio OCR, y el modelo de base de datos.” Cursor genera los 3 archivos coherentes entre sí.
3. **Tab completion:** Cursor predice el siguiente código basado en el contexto. Escribe 3 líneas, Cursor sugiere las siguientes 5. Eso acelera la iteración 2x-3x.
4. **Narrativa de la entrevista:** “Usé Cursor como copiloto, no como reemplazo. Le pedí sugerencias, revisé cada línea, y entendí por qué funcionaba. Así puedo explicarlo en la entrevista.”

8. Kimi Claw — El Asistente Automático

¿Por qué Kimi Claw?

Kimi Claw es el agente de automatización de Kimi. Lo usamos para:

- Generar la estructura de carpetas inicial
- Crear los templates de PR

- Documentar decisiones técnicas
- Revisar código antes de commitear

Justificación

1. **Automatización de boilerplate:** No perdemos tiempo escribiendo `.gitignore`, `README.md` inicial, o estructura de carpetas. Kimi Claw lo genera en segundos y nosotros lo revisamos.
2. **Consistencia:** Cada PR tiene el mismo formato, cada commit sigue convenciones, cada archivo tiene el mismo estilo. Eso es profesionalismo.
3. **Narrativa de la entrevista:** “Uso IA para acelerar el boilerplate, pero el código crítico (prompts, validaciones, lógica de negocio) lo escribo y reviso yo. La IA me ayuda a ir más rápido, no a pensar menos.”

9. Git + GitHub — Control de Versiones

¿Por qué GitHub Flow?

1. **Ramas por tarea:** Cada tarea del Kanban tiene su rama. `backend/upload-endpoint`, `frontend/dropzone-ui`. Eso es limpio, trackable, y fácil de revisar.
2. **PRs como documentación:** Cada PR tiene título, descripción, checklist, y criterios de aceptación. Un evaluador puede leer la historia del proyecto solo leyendo los PRs.
3. **GitHub Projects como Kanban:** Backlog → En Progreso → Review → Done. Visible para todo el equipo (aunque seamos uno solo, demuestra que sabemos organizar trabajo).
4. **Narrativa de la entrevista:** “Mi flujo de trabajo es: tarea del Kanban → rama → desarrollo → PR → review → merge. Así cada cambio está documentado y reversible.”

 Comparativa: Nuestro Stack vs Alternativas

Capa	Nuestra elección	Alternativa común	Por qué ganamos
LLM/ OCR	Groq (Llama 4 Scout)	OpenAI GPT-4o (pago)	Gratis, 30x más rápido, OpenAI-compatible
LLM/ OCR (alternativa)	Groq (Llama 4 Scout)	GitHub Models (GPT-4o)	30x más rápido, misma gratuidad, impresiona en demo
Backend	FastAPI	Flask	Async nativo, docs auto, Pydantic integrado
Frontend	Next.js 14 + shadcn/ui	React vanilla	Componentes profesionales, file routing, Tailwind
Base de datos	SQLite	PostgreSQL	Zero-config, portable, suficiente para demo
Estructuración	Pydantic (parseo manual)	Instructor + OpenAI	Más control, logging explícito de fallos del LLM

Table 7 – continued

Capa	Nuestra elección	Alternativa común	Por qué ganamos
Validación	Reglas propias	Confianza ciega en LLM	Detectamos errores, no los escondemos
IDE	Cursor	VS Code	IA integrada, composer, tab completion

🎙 Narrativa para la Entrevista de 30 Minutos

“Elegí Groq con Llama 4 Scout porque es 30 veces más rápido que la API estándar de OpenAI, y completamente gratuito. En una demo de 30 minutos, la velocidad importa: el usuario sube una nómina y ve el resultado en menos de un segundo. Eso no es magia, es arquitectura.”

“Para el OCR, no uso Tesseract puro porque las nóminas españolas tienen formatos variados, tablas, y campos dispersos. Un modelo de visión como Llama 4 Scout entiende el contexto visual. Pero no confío ciegamente en el LLM. Diseñé un parser Pydantic con reintentos automáticos, y luego valido con reglas de negocio: ¿el IBAN tiene 24 dígitos? ¿los netos son menores que los brutos? ¿la fecha es válida?”

“Cuando el validador detecta un error, no rechaza todo. Marca el campo como confidence: low, lo resalta en naranja en el frontend, y el usuario corrige solo lo que está mal. Eso es UX de producto real.”

“Todo el código está en GitHub con PRs limpios, cada tarea tiene su rama, y el README documenta cada decisión técnica. Un tercero puede clonar, instalar y probar en 5 minutos.”

✅ Conclusión

Cada tecnología del stack fue elegida con dos criterios:

1. **Funcional:** ¿Resuelve el problema de la mejor manera posible?
2. **Narrativa:** ¿Puedo explicar por qué la elegí en una entrevista de 30 minutos?

Groq (Llama 4 Scout) gana en ambos: es funcionalmente superior (gratis, ultra-rápido, OpenAI-compatible) y narrativamente potente (la velocidad impresiona en demo, demuestra que pensamos en experiencia de usuario). El resto del stack (FastAPI, Next.js, SQLite, Pydantic) forma un ecosistema coherente donde cada pieza se refuerza con las demás.

El resultado no es un proyecto más. Es un producto que demuestra cómo trabajamos con IA, cómo manejamos sus fallos, y cómo construimos con calidad de producción.

“No buscamos un entregable perfecto. Queremos entender cómo trabajas, cómo planteas un problema, cómo lo construyes y cómo lo desarrollas.” — Miguel Álvarez Mulas, Kontaktu AI